czii-cryo-et-object-identification

2位解法

Rank	2
Team	LuoZiqian&Lion
Author	AnnieGo
Topic ID	561568
Version	Japanese Translation

Source: https://www.kaggle.com/competitions/czii-cryo-et-object-identification/discussion/561568
Retrieved with Kaggle CLI on 2026-10-21.

Kaggle Discussionの主投稿と、技術的補足を含む選定済みQ&Aを日本語へ翻訳した版です。code、URL、model名、識別子、数値は原文を保持しています。

謝辞

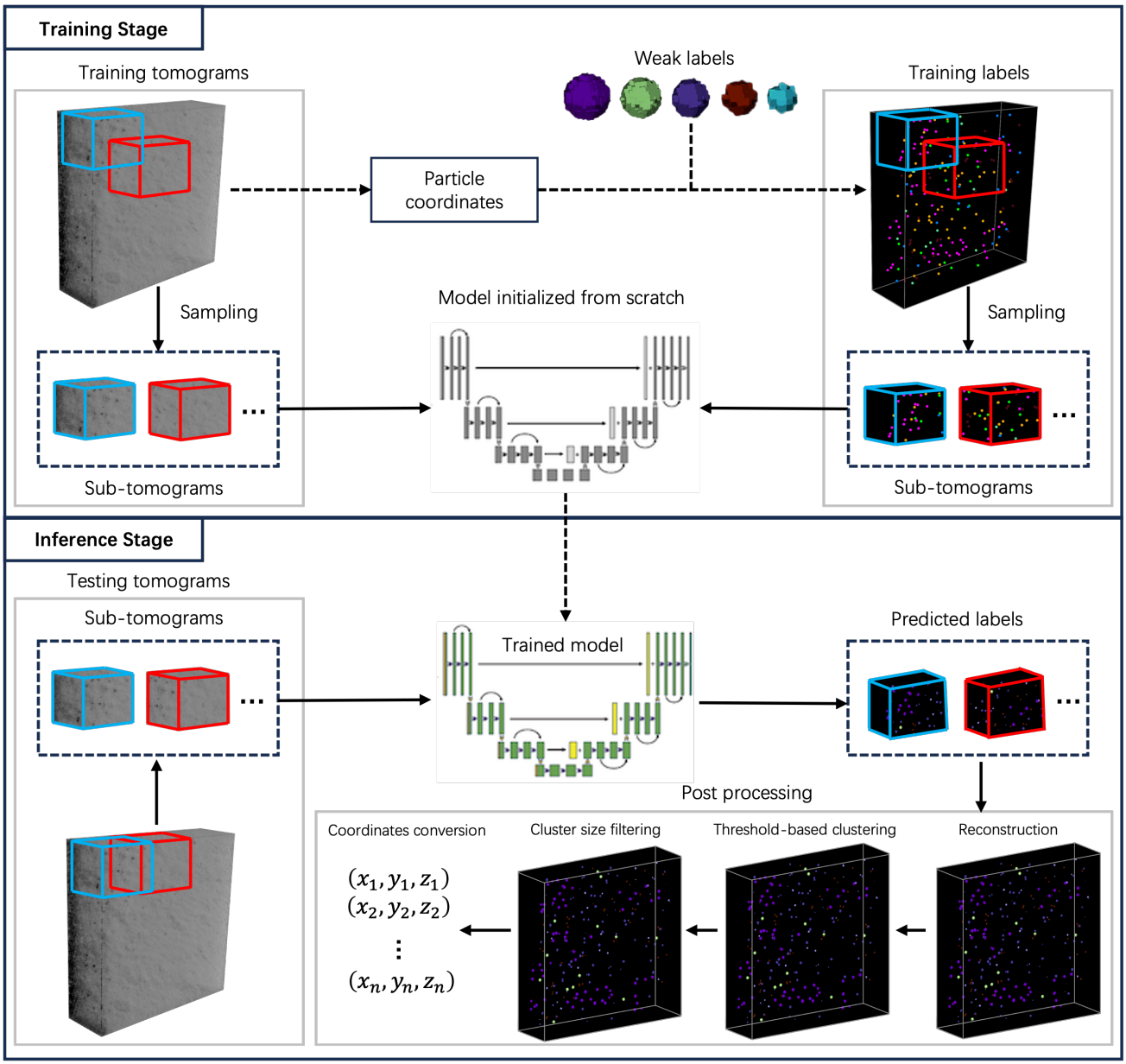
この貴重な機会を提供してくださった Kaggle とコンペ主催者に心から感謝します。また、多大な貢献をしてくれた @hengck23 と @fnands にも感謝します。fnands の notebook と hengck23 の discussion は、私たちのアプローチの確かな土台となりました。最後に、すばらしい協力をしてくれたチームメイトの @luoziqian に感謝します。

概要

私たちのアプローチは、パラメータ数が 873K から 14.2M までの、複数の軽量なセグメンテーションモデルのアンサンブルに基づいています。セグメンテーションの後、CC3D を使って粒子の重心を計算し、ボクセル数の統計に基づいて小さなクラスタをフィルタしました。

全体のパイプライン

全体のパイプラインを下に示します。

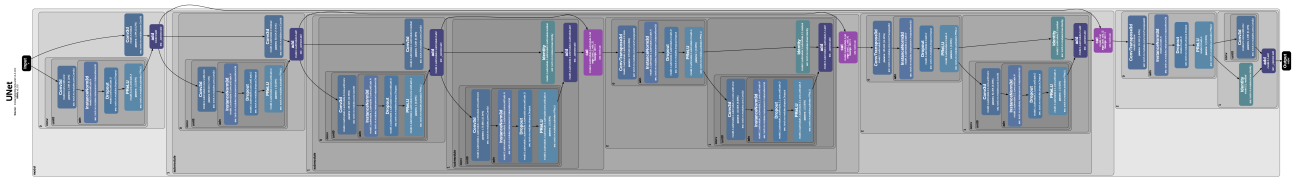


モデルのアーキテクチャ

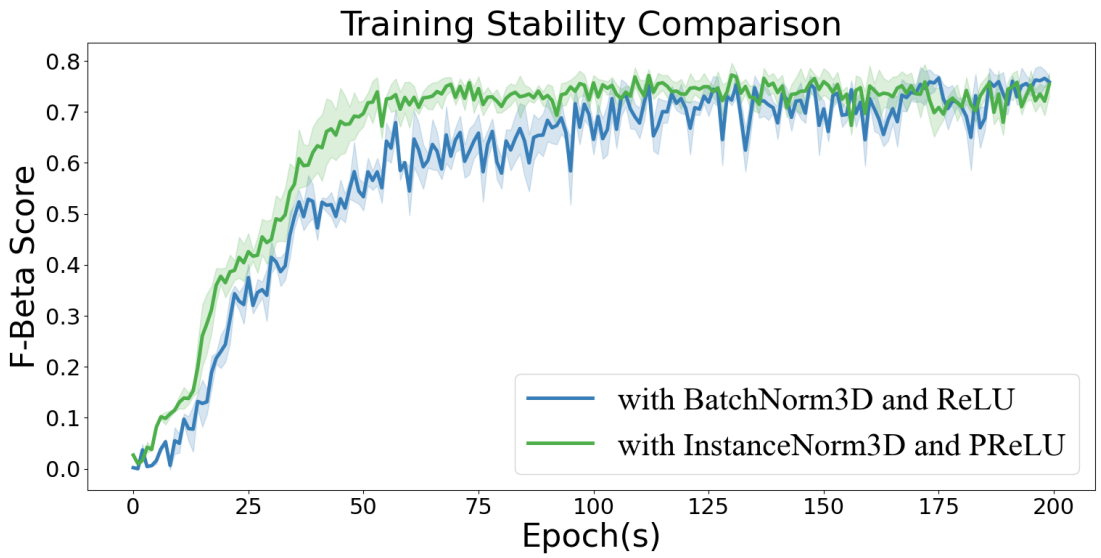
実験は MONAI UNet のベースラインモデルから始めました。試行を通じて、パラメータ数の多いモデルは過学習しやすく、軽いモデルより性能が劣ることが多いと分かりました。そこで、UNet3D、VoxResNet、VoxHRNet、SegResNet、DynUNet、DenseVNet といった軽量なアーキテクチャを選びました。

ただし、VoxResNet、VoxHRNet、DenseVNet で実験したところ、性能の揺れや収束の難しさといった安定性の問題に直面しました。さらに分析したところ、MONAI UNet は InstanceNorm3d と PReLU を使っていることが分かりました。それに合わせて正規化層と活性化層を変更したところ、モデルの性能がより安定しました。

MONAI UNet のモデルアーキテクチャを下に示します。



次の図は、InstanceNorm3d と PReLU を使った MONAI UNet と、BatchNorm3d と ReLU を使ったものの学習性能を、5つの実験で比較したものです。InstanceNorm3d と PReLU を使うと、学習がより安定することが分かります。



最終アンサンブルには、public leaderboard のスコアに基づいて次のモデルを選びました。

- UNet3D
- VoxResNet
- VoxHRNet
- SegResNet
- DenseVNet
- UNet2E3D

学習戦略

学習の設定は、安定性と最適な性能を確保するように設計しました。セグメンテーションのマスクの半径、loss 関数、データ拡張の戦略が、信頼できる結果を得るうえで決定的な役割を果たすと分かりました。

セグメンテーションのマスク

各粒子について、カスタマイズした半径の正解マスクを使いました。

粒子の種類	デフォルトの半径	Luoziqian の半径	Lion の半径
apo-ferritin	60	60 x 0.5	80 x 0.4

粒子の種類	デフォルトの半径	Luoziqian の半径	Lion の半径
beta-galactosidase	90	90 x 0.5	90 x 0.4
ribosome	150	150 x 0.5	150 x 0.4
thyroglobulin	130	130 x 0.5	120 x 0.4
virus-like-particle	135	135 x 0.5	150 x 0.4

学習の設定

Lion のモデル:

- patch size [128, 256, 256] または [128, 384, 384]
- 200 エポック
- 7 クラス
- Optimizer: 学習率 0.001 の AdamW（学習率スケジューラなし）
- バッチサイズ: 2 または 4
- Drop path rate: 0.1 または 0.3
- F-beta 指標に基づくモデル選択。上位5つの最良モデルを保存
- Loss 関数:
  - Tversky Loss
  - 重み [1.0, 1.0, 0.0, 2.0, 1.0, 2.0, 1.0] の Cross-Entropy Loss

Luoziqian のモデル:

- patch size [128, 200, 200] または [128, 256, 256]
- 100 または 300 エポック
- 6 クラス
- patience=20 の Early stopping
- バッチサイズ: 1 または 2
- モデル選択は評価 loss に基づく。最後の 100 モデルを保存し、その F-beta スコアを確認する
- Loss 関数:
  - Dice Loss
  - Tversky Loss
  - 0.05 または 0.1 にスケールした Cross-Entropy Loss

データ拡張

Lion のモデル:

- バランスの取れたサンプリングのための、比率 [1, 1, 1, 1, 2, 1, 2] の RandCropByLabelClassesd
- RandRotate90d
- RandFlipd
- RandAffined

- RandGaussianNoised

Luoziqian のモデル:

- RandCropByLabelClassesd
- RandRotate90d
- RandFlipd
- RandShiftIntensityd

7 TTA によるモデル性能のまとめ（一部抜粋）

No	Model	開発者	アーキテクチャ	パラメータ数	Valid ID	正規化	活性化	Public LB	Private LB
1	epoch122-step2952-valid_loss0.3625-val_metric0.8367.ckpt	Lion	UNet3D	1.1M	TS_86_3	InstanceNorm3d	PReLU	0.77379	0.76582
2	epoch148-step3576-valid_loss1.1154-val_metric0.7722.ckpt	Lion	UNet3D	1.1M	TS_6_4	InstanceNorm3d	PReLU	0.77021	0.76725
3	epoch153-step3696-valid_loss0.3021-val_metric0.8900.ckpt	Lion	UNet3D	1.6M	TS_69_2	InstanceNorm3d	PReLU	0.77205	0.76676
4	epoch194-step4680-valid_loss1.0213-val_metric0.8788.ckpt	Lion	UNet3D	1.1M	TS_69_2	InstanceNorm3d	PReLU	0.77390	0.76737
5	epoch138-step3336-valid_loss0.3690-val_metric0.8476.ckpt	Lion	UNet3D	1.1M	TS_73_6	InstanceNorm3d	PReLU	0.76543	0.76025
6	epoch152-step3672-valid_loss0.4333-val_metric0.7929.ckpt	Lion	DenseVNet	873K	TS_6_6	InstanceNorm3d	PReLU	0.76528	0.75417
7	epoch195-step4704-valid_loss0.4258-val_metric0.7914.ckpt	Lion	VoxResNet	7.0M	TS_6_6	InstanceNorm3d	PReLU	0.77457	0.76593
8	epoch188-step4536-valid_loss0.4231-val_metric0.8659.ckpt	Lion	VoxHRNet	1.4M	TS_73_6	InstanceNorm3d	PReLU	0.76738	0.75995
9	epoch198-step4776-valid_loss0.3471-val_metric0.8730.ckpt	Lion	VoxHRNet	1.4M	TS_73_6	InstanceNorm3d	PReLU	0.76135	0.75848
10	epoch133-val_loss0.52-val_metric0.56-step3216.ckpt	Luoziqian	UNet3D	1.1M	TS_6_4	BatchNorm3d	PReLU	0.76844	0.76320

No	Model	開発者	アーキテクチャ	パラメータ数	Valid ID	正規化	活性化	Public LB	Private LB
11	epoch314-val_loss0.54-val_metric0.54-step7560.ckpt	Luoziqian	SegResNet	1.2M	TS_6_4	GroupNorm	ReLU	0.75521	0.74647
12	epoch114-val_loss0.55-val_metric0.53-Step2760.ckpt	Luoziqian	UNet2E3D	14.2M	TS_6_4	BatchNorm3d	ReLU	0.73758	0.72966

アンサンブル戦略（一部抜粋）

最終提出には、public leaderboard のスコアが最も良いモデルを選びました。

Models	アンサンブル戦略	確信度の閾値	Public LB	Private LB	選択
[1, 2, 3, 4, 7, 8, 12]	Average	0.23	0.79094	0.78641	❌
[1, 2, 3, 4, 7, 8, 12]	Average	0.18	0.79213	0.78630	❌
[1, 2, 3, 4, 7, 8, 12]	Average	0.15	0.79307	0.78457	❌
[1, 2, 3, 4, 7, 8, 12]	weighted	0.15	0.79247	0.78417	❌
[1, 2, 3, 4, 6, 12]	Average	0.15	0.78701	0.78389	❌
[1, 2, 3, 4, 5, 6, 7]	Average	0.15	0.79104	0.78277	❌
[1, 2, 3, 4, 7, 9, 12]	Average	0.15	0.79391	0.78283	✅
[1, 2, 3, 4, 6, 7, 9, 10, 11, 12]	Average	0.15	0.79355	0.78381	✅

後処理（クラスタの除去）

CC3D を使ってセグメンテーション結果を粒子のクラスタに変換し、クラスタの選択に次の戦略を適用しました。

- 閾値に基づくクラスタリング（recall を高めるため、閾値はすべてのクラスで 0.15 に設定）
- クラスタのサイズによるフィルタリング

改善につながらなかった手法

- 合成データ
- 2段階目の分類
- 重いモデル（transformer ベースのモデル）

コード

推論 Notebook

Lion の学習コード

Luoziqian の学習コード

参考文献

Gubins, Ilja, et al. "SHREC 2020: Classification in cryo-electron tomograms." Computers & Graphics 91 (2020): 279-289.

補足Q&A

FatLiuyun — 質問

おめでとうございます！解法を共有していただきありがとうございます。

質問があります。なぜ実験 no.2 と no.10/11/12 は、valid のスコアがこれほど大きく違う（0.7x 対 0.5x、でも lb はとても近い）のでしょうか。

AnnieGo — 回答

「0.7x vs. 0.5x, but lb very close」が何を意味するのか、もう少し説明していただけますか。なお、ここまでに示した実験は、全実験結果のごく一部にすぎません。また、同僚の Luoziqian は MONAI UNet のアーキテクチャに基づいて public leaderboard のスコアが 0.760 を超えるモデルをいくつか開発していますが、それらはアンサンブルしても性能の向上が見られません。

FatLiuyun — 質問

（原文は中国語）ああ、こういうことです。あなたの実験の model 列に val_metric という項目がありますが、no.2 と no.10、no.11、no.12 はどれも TS_6_4 を検証セットにしています。これらの検証セットの指標の差がかなり大きいようなので、何か特別な処理を私が見落としているのか伺いたいと思いました。

AnnieGo — 回答

私たちの実験では、空間サイズ、マスクの半径、データ拡張の戦略、loss 関数、正規化層、活性化層など、いくつかの要因が性能に大きく影響することが示されました。

実験 No.2 と実験 No.10 の比較では、次の違いが影響していることが観察されました。

- パッチの空間サイズ
- マスクの半径
- データ拡張の戦略
- loss 関数
- クラス数

さらに、実験 No.2 と実験 No.11、No.12 を比べると、MONAI UNet が SegResNet と UNet2E3D の両方を上回りました。この性能向上は、MONAI UNet のいくつかのアーキテクチャ上の特徴によるものと考えられます。

- InstanceNorm3d と PReLU 活性化の組み合わせ
- 1/8、1/16、1/32 ではなく 1/4 のエンコーダのダウンサンプリング係数
- 通常のアップサンプリングではなく、ConvTransposed3D を使った deconvolution 層の実装
- パラメータ効率を高め、過学習の可能性を減らす residual block の導入

SegResNet と UNet2E3D の両方について、学習の収束を助けるために、実験的な調査に基づいて変更を加えました。具体的には、次のとおりです。

- 正規化層と活性化層を見直した
- デコーダを deconvolution 層に置き換えた
- デコーダのチャネル数を減らし、より軽量なモデルにした

これらの変更にもかかわらず、SegResNet も UNet2E3D も MONAI UNet を上回りませんでした。

ForcewithMe — 質問

おめでとうございます！ @sjtuwangshuo。

検証にはどのサンプルを使っていますか。

AnnieGo — 回答

最初の戦略:

- トモグラムを7つの fold に分割する（例: TS_5_4、TS_6_4、TS_69_2 など）。
- 各 fold を学習し、F-beta スコアが最も高い上位5つのモデルを保存する。
- 上位5つのモデルそれぞれを、同じ検証サンプルに対して 7× Test-Time Augmentation (TTA) で評価し、F-beta スコアを計算する。
- 7× TTA 後の F-beta スコアが最も高いモデルを提出用に選ぶ。

改良:

その後、Luoziqian は、['TS_5_4', 'TS_6_4', 'TS_69_2'] で検証したモデルの方が、常にではないものの public leaderboard での性能が良い傾向があることに気づきました。この観察に基づき、私はこれらの特定の fold で 7× TTA を使ってモデルの性能を評価し、F-beta スコアが最も高いモデルを提出しています。

最終アンサンブルで使ったモデルの検証サンプルの詳細については、この投稿の「7 TTA によるモデル性能のまとめ（一部抜粋）」の節を参照してください。

Aurora_blue — 質問

おめでとうございます！

segmentation + cc3d のアプローチでも改善の余地はたくさんあること、そして実験を設計する私のスキルが足りないことに気づきました。

Luoziqian の学習コードの中に、コンペ指標を出力する部分が見つかりません。別の方法で最良モデルを選んだのですか。

LuoZiqian — 回答

compute-cv-7TTA.ipynb の path_dir を変更して実行すれば、学習後の ckpt の F-beta4 スコアを計算できます。最初にうまくいかない実験を除外するには、各クラスの Tversky Loss で十分だと考えています。学習中に F-beta4 を測ると実験時間が大幅に増え、計算資源も限られていたので、学習中には計算しませんでした。